TechZone — Software Requirements Specification

Team 9 · Revision 2.0 · 11 November 2026 · Implementation baseline for Part 3

1. Introduction and document control

This revision follows the section structure of the team's original IEEE-style SRS. It specifies the delivered internal electronics-store system and retains the original requirement identifiers for traceability. It supersedes the original implementation descriptions for this delivery, while the original root PDFs remain submission history. This is a team revision, not a claim of faculty approval, formal IEEE certification or acceptance of unmet requirements.

Prepared under Atharv Mittal's requested direct-main documentation/QA delivery. Module owners must review the changes: Annamalai (authentication/security), Apurv (catalogue/stock), Avaneesh (inventory/sales/reports), Atharv (foundation/suppliers/purchases/common/NFR). Outstanding acceptance items are explicit below.

The original design used React, Django REST Framework, PostgreSQL, Jira, Jenkins, pytest and six sprints. The delivered system uses React/Vite/Tailwind, Express/Node.js ES modules, MongoDB/Mongoose, GitHub Issues/Projects, GitHub Actions and Node's built-in test runner with Supertest/jsdom. Sprint 1 ran 28 October–4 November and Sprint 2 ran 5–9 November 2026. The Part 3 deadline was 6 November; catch-up target is 12 November 2026 EOD IST. No earlier deadline has been rewritten as met.

References: original `Team_9_TechZone_Electronics_SRS.pdf`; Parts II/III of `Team_9_TechZone_Final_Project_Documentation.pdf`; SAD; Test Plan; test case execution; implementation decisions.

2. Purpose, scope and system context

TechZone is an internal staff application for product/category maintenance, available stock, adjustments, suppliers, purchase orders/receipts, sales and inventory/sales reporting. It supports five application roles: ADMIN, INVENTORY, PURCHASE, SALES and MANAGER. It is not a customer storefront and has no online payment, tax accounting, shipping, barcode integration or multi-store workflow.

The web client uses a staff login followed by separate Inventory, Products, Suppliers, Purchases, Sales, Reports and Team pages, limited by permissions. The backend alone accesses the database. Administrative user management is limited to read-only listing; accounts are provisioned using controlled development seed/model operations. General inventory-history filtering and dedicated aggregate purchase reporting remain baseline gaps rather than completed features.

3. Users and operating environment

Staff use a desktop browser; Chromium desktop and mobile-width workflow checks are recorded. Firefox and Edge remain required manual checks. Runtime requires Node.js >=24, npm workspaces and a MongoDB replica set for transactions. The verified machine uses Arch Linux, Node 26.12.1 and MongoDB 8.3.9. Native Windows/macOS CI setup exists; this run does not assert a new test execution on those hosts.

Local development uses Vite at localhost:5173, an Express API on 127.0.0.1:5000 and a project MongoDB replica set on loopback port 27018. QA uses disposable named databases; this delivery's isolated replica set is on port 27019. Production TLS, authenticated DB users, deployment monitoring and backups are outside the verified local environment.

4. External interfaces and constraints

The client sends JSON requests under `/api`; Vite proxies them to the server during development. Authentication uses an opaque `tz_session` cookie with HttpOnly, SameSite=Strict and `/api` path; Secure is enabled in production mode. Session validity is eight hours, with database expiry checked on each request. Only a SHA-256 hash of each random session token is stored.

Business routes enforce authentication and named server permissions. The client hides unavailable actions but cannot grant authorization. Errors use `{ error: { message } }`; ordinary responses use named record collections or objects. Codes include 400 invalid input, 401 no valid session, 403 forbidden, 404 absent record, 409 business conflict and generic 500 server failure. The API limits JSON bodies to 100 KB.

Money uses nonnegative safe integers in minor units, displayed to two decimal places. Quantities use safe integers. Dates use ISO values; `from`/`to` filters use inclusive UTC calendar dates. IDs are MongoDB ObjectIds. Master-record retirement preserves references and transaction snapshots. No client-side database credential is required.

5. Use cases

ID	Actor / workflow	Delivered status
UC-01	All roles: login, use permitted pages, logout	Implemented; API and browser verified
UC-02	Admin: manage users/roles; Manager: view users	Partial: role enforcement and read-only Team page; account mutation UI/API deferred
UC-03	Admin/Inventory: maintain catalogue/categories	Implemented; only Admin retires products

UC-04	Inventory/Admin: view, adjust and monitor stock	Implemented except general history browsing/filtering
UC-05	Purchase/Admin: maintain suppliers	Implemented
UC-06	Purchase/Admin: create/edit unreceived order	Implemented
UC-07	Purchase/Admin: receive ordered goods	Implemented; partial/full receipts, no over-receipt
UC-08	Sales/Admin: record sale	Implemented; exact stock and money, safe retries
UC-09	Manager/Admin and permitted specialist staff: reports	Inventory/low-stock/sales implemented; purchase history substitutes only part of purchase reporting

6. Functional requirements

“Implemented” describes current behavior. Deferred/partial rows retain unmet original scope and require a team decision; they are not silently removed acceptance criteria.

6.1 Authentication and users

Requirement	Current specification / status
FR-AUTH-001	Active registered staff can log in with valid credentials at POST /api/auth/login. Implemented.
FR-AUTH-002	Invalid credentials return a generic 401 and create no session. Implemented.
FR-AUTH-003	Inactive accounts cannot log in; deactivation also ends existing access. Implemented.
FR-AUTH-004	POST /api/auth/logout terminates the session. Implemented.

FR-AUTH-005	Each active user has one of five roles; stored role changes apply on the next request. Implemented at model/authorization level.
FR-AUTH-006	Admin-only create/update/deactivate/reactivate account workflow. Deferred: no mutation routes or UI.
FR-AUTH-007	Server permission guards authorize every business operation. Implemented.
FR-AUTH-008	Disallowed roles receive access denied. Implemented.

6.2 Products and categories

Requirement	Current specification / status
FR-PROD-001	Create SKU/name/category/selling price/minimum stock and initial zero-stock row atomically. Implemented.
FR-PROD-002	Reject duplicate normalized SKU on create/update; uniqueness includes retired SKUs. Implemented, stricter than active-only baseline.
FR-PROD-003	Validate and persist product edits. Implemented.
FR-PROD-004	Maintain categories and reject deletion of a referenced category. Implemented.
FR-PROD-005	Search by SKU/name through the Inventory page/API; category search/filter also exists. Implemented with relocated UI.
FR-PROD-006	Retire products without deleting stock/history. Implemented as logical DELETE.

6.3 Inventory

Requirement	Current specification / status
FR-INV-001	Show stored current quantity for active products, with search/category/low-stock filters. Implemented.

FR-INV-002	Authorized signed adjustment records product, reason, actor and time. Implemented.
FR-INV-003	Reject adjustments producing negative stock; write no orphan history. Implemented.
FR-INV-004	Low stock means availableQty <= minimumStock, including equality. Implemented.
FR-INV-005	Persist immutable purchase/sale/adjustment history with source and actor. Implemented; verified at database/service level.
FR-INV-006	General product/date inventory-history filtering. Deferred: indexes/history exist, but no general API/UI.

6.4 Suppliers and purchases

Requirement	Current specification / status
FR-PUR-001	Purchase/Admin can create/edit supplier name and optional contact/phone/email/address. Implemented.
FR-PUR-002	Supplier deactivation preserves purchase records/snapshots. Implemented.
FR-PUR-003	Create order for active supplier and active products, with no stock movement at ordering. Implemented.
FR-PUR-004	Require 1–100 unique product lines, positive whole ordered quantity, nonnegative unit cost. Implemented.
FR-PUR-005	Derive each line and order total from integer quantity/cost. Implemented.
FR-PUR-006	Record partial/full received quantities against an existing ordered purchase. Implemented; added missing TC-PUR-06.
FR-PUR-007	Receipt, received progress, stock and history commit atomically. Implemented.
FR-PUR-008	Reject quantity exceeding remaining order balance, including concurrent receipt attempts. No exception process. Implemented.

FR-PUR-009	Filter history by supplier/inclusive dates/status; retain supplier snapshot and receipt details. Implemented.

Order states are DRAFT, ORDERED, PARTIAL, RECEIVED and CANCELLED. Unreceived orders can be edited/cancelled subject to state guards; received lines cannot be rewritten. New orders require active suppliers; an existing order can still receive after supplier deactivation. Creation/receipt retries require a stable `Idempotency-Key`; conflicting reuse fails. See purchase contracts for limits and transitions.

6.5 Sales

Requirement	Current specification / status
FR-SALE-001	Sales/Admin can create one sale with valid product lines. Implemented.
FR-SALE-002	Require positive whole safe-integer quantities. Implemented.
FR-SALE-003	Reject any line exceeding available stock without partially completing the sale. Implemented.
FR-SALE-004	Derive totals using current stored product prices, snapshotting exact minor units. Implemented.
FR-SALE-005	Decrement stock, append history and persist sale atomically; retry does not deduct twice. Implemented.
FR-SALE-006	Assign unique transaction number protected by database index. Implemented.
FR-SALE-007	Filter sales by transaction number and inclusive date range. Implemented.

6.6 Reports and common behavior

Requirement	Current specification / status
FR-REP-001	Inventory report includes SKU/product/category/quantity/threshold/stock status, counts and total units. Implemented.
FR-REP-002	Low-stock report includes active products at/below minimum. Implemented.

FR-REP-003	Supplier/date purchase report. Partial: purchase-history filtering/per-order totals exist; dedicated endpoint, aggregate totals and report UI are absent.
FR-REP-004	Sales report supports dates, transaction count, units sold and total money. Implemented.
FR-REP-005	Inventory/sales report values derive from stored transactions. Implemented for delivered reports; purchase reporting remains partial above.
FR-COM-001	Validate required inputs before writes. Implemented using server guards and client forms; messages may be form-level rather than per-field.
FR-COM-002	Provide actionable invalid-input/operation messages. Implemented and tested for representative forms/errors.
FR-COM-003	Confirm successful mutations; distinguish committed writes from failed refreshes. Implemented.
FR-COM-004	Show explicit no-match messages with filter reset. Implemented.
FR-COM-005	Preserve references/history when records change or retire. Implemented through application checks, transactions, snapshots, validators and indexes.

7. Non-functional requirements and verification boundaries

ID	Original target / current validation
NFR-001	>=95% normal authenticated requests <=2 seconds under defined load. Tested with 100 sequential warmed reads across five routes on a small local database; no production or concurrency claim.
NFR-002	>=99% availability over a defined operating window. Unverified/N/A locally: no deployed window/monitor.
NFR-00	Trained user can find product/stock within three primary screens. Inventory provides search and quantity on one page; browser search/clear smoke passes. No

3	original dedicated TC ID exists.
NFR-004	Retry creates no duplicate purchase/sale transaction. Real concurrent/repeated API/service and frontend retry checks.
NFR-005	Separated business modules with core tests. Module boundaries and test inventory reviewed.
NFR-006	Deterministic API/service behavior suitable for testing. Unique disposable databases and controlled fault injection; no skipped integration checks in the recorded run.
NFR-007	Current Chrome/Chromium, Firefox and Edge support. Chromium desktop/mobile-width verified; Firefox and Edge pending.
NFR-008	>=10000 products and 100000 transaction/item records without redesign. Synthetic 100000 valid audit records and 10000 products tested; three representative operations have a local 5-second usability budget, separate from NFR-001.
NFR-009	Atomic purchase/sale/inventory updates. Forced later-line and receipt-persistence failures leave no partial changes.
NFR-010	Messages identify corrective field/operation. Representative React and API-client error assertions.
NFR-011	Naming/modularity/static analysis/Git review practices. ESLint/build checks pass; direct-main delivery is user-authorized, so peer review remains pending.
NFR-012	Inventory events retain responsible actor and timestamp. Database/service audit assertions.

8. Security requirements

ID	Current design / validation
SEC-001	Protected routes require valid session. API regression tests.
SEC-002	Salted scrypt password hashing; password fields omitted from public users; hashed session tokens. Model/auth tests.
SEC-003	Named server-side permissions; every role/operation matrix tested.
SEC-004	Forged client roles do not alter authorization. Negative API tests.
SEC-005	HTTPS required for production and Secure cookies enabled in production mode. Production network verification N/A without deployment.
SEC-006	Validate types, lengths, IDs, numeric limits, unknown fields and literal search. Negative tests.
SEC-007	Server-only secrets; ignored .env; client source/build inspection. Snapshot review, not a historical secret audit.
SEC-008	Same failed-login status/message for unknown, wrong-password and inactive accounts; unknown users use a decoy hash. No timing indistinguishability claim.
SEC-009	Inactive users cannot authenticate or retain sessions. Tests.
SEC-010	Least-privilege authenticated database users required for deployment. Local helper has no authentication and is loopback-only; deployed privilege inspection N/A.

9. Business rules and logical data

Rule	Current invariant
BR-001	Unique normalized SKU, including retired records

BR-002	Available quantity never negative
BR-003	Low stock includes equality to minimum
BR-004	Only active suppliers on new purchase orders
BR-005	Purchase has at least one valid item
BR-006	Only received goods increase stock
BR-007	Completed sale decreases stock by exact sold quantity
BR-008	No sale above available stock
BR-009	Preserve referenced master/history records by deactivation
BR-010	Authorized roles alone change protected records
BR-011	Derive money totals from quantities and stored costs/prices
BR-012	Inventory-affecting events record source, actor and time

Physical collections: users, sessions, categories, products, inventory, inventoryTransactions, suppliers, purchases, purchaseReceipts and sales, plus schema migrations. Embedded purchase/sale lines snapshot product/SKU/name/cost; purchases snapshot supplier contacts. Stock is one unique row per product. History is append-only through model/service contracts. MongoDB validators and indexes protect direct document values and uniqueness; services enforce cross-collection references and transaction boundaries. Details: database.

10. Acceptance, traceability and outstanding decisions

Acceptance evidence is the revised Test Plan, four QA reports, all 65 original case IDs plus added TC-PUR-06, and the recorded source manifest. A test pass is evidence for its documented scope, not proof that deferred baseline scope is delivered.

Outstanding decisions: resolve FR-AUTH-006, FR-INV-006 and FR-REP-003 with module owners; complete Firefox/Edge checks and peer review; retain production-only N/A dispositions until a deployment exists. No approval/waiver is inferred from this revision. Atharv will upload the already-recorded Sprint 1 video and later record/upload Sprint 2. QA catch-up and demo preparation are tracked in delivery checklist.